Learning from Earlier Websites: Cross-Task Long-Term Memory for a Small Website-Generation Model



Qiaopan Ma

Supervisor:
Qian Liu

School of Computer Science
The University of Auckland

A dissertation submitted in partial fulfilment of the requirements for the degree of
Master in Artificial Intelligence,
the University of Auckland, 2026.

This dissertation is for examination purposes only and is confidential to the examination process.

Abstract

Website-generation agents solve every task from scratch, forgetting what went wrong on earlier websites. This dissertation asks whether a system built around a *small* generator, WebGen-LM-7B, can improve across a sequence of WebGen-Bench tasks when it is given a long-term memory built from its own earlier attempts, without changing its weights and while retaining the official single-attempt generation and evaluation of each task. A separate, larger model maintains the memory.

We reproduce the official No Memory baseline on a self-hosted cluster (28.5% accuracy on 101 tasks; 28.4% reported). Its failures show that many zero-score tasks are blank pages caused by basic code errors, such as importing a package that is not installed, and, in a nine-task diagnostic, short textual warnings did not reliably prevent these failures. A text-dominated memory improved early performance but later fell behind No Memory; the experiments do not isolate the cause of this decline.

Code memory adds a requirement checklist and a code pool: when earlier tasks show a code error that breaks a site, the prompt carries a short fragment copied verbatim from the chain's own compiled, successful websites. In a single run over all 101 tasks it reaches 40.0% against 28.5% (paired gain 11.5 points, 95% CI 6.0–16.9) and cuts zero-score tasks from 30 to 13; a checklist-only ablation gains 3.4 points, suggesting that the stored cross-task content accounts for most of the improvement in this run. Under shuffled orders, a whole-file example made the generator return the untouched starter project. The final method shows only verified fragments; no scaffold returns were observed in either shuffled-order run, with gains of 7.4 and 5.5 points. [TODO: Add the final method on the manifest order.] Gains are functional, largely from sites that now render; appearance on sites working in both arms is similar.

The main observation is that, for this small generator, the *form* of memory mattered: short code fragments reduced the targeted failures where textual warnings did not reliably do so. All results are single runs on one benchmark on which the method was also developed; we discuss these limits, the cost of the larger memory model, and ethical considerations. [TODO: Status: fixed-prefix ablation queued; its result decides whether the gain is described as accumulated memory or as automatically discovered prompt code.]

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, Dr Qian Liu, for supervising this project, for giving me the freedom to pursue a research direction I cared about, and for the support, encouragement and opportunities to present and discuss the work at group meetings. I am grateful for the trust and the research environment provided throughout this year.

I am also deeply grateful to Ziqin Zhu, a PhD student in Dr Liu's group, for generous guidance on how to do research and how to write it up: how to frame research questions, design fair experiments, and structure a dissertation. This advice shaped much of the way this work was carried out and written.

I also thank Tanya Carter, the programme manager of the Master of Artificial Intelligence, for coordinating the research project course, including the supervision arrangements.

I am grateful to the School of Computer Science at the University of Auckland for access to the CS ML GPU cluster. All experiments in this dissertation were run on its NVIDIA H200 GPUs; without this resource, running more than a thousand website generations with self-hosted models would not have been possible.

Finally, I thank my family for their support.

Qiaopan Ma
October, 2026

Table of Contents

List of Figures **v**

List of Tables **vii**

1 Introduction **1**

1.1 Background 1
1.2 Problem Statement 2
1.3 Research Questions 3
1.4 Scope 3
1.5 Contributions 4
1.6 Organisation of Thesis 4

2 Background and Related Work **5**

2.1 Website Generation Benchmarks 5
2.2 Memory for LLM Agents 6
2.3 Small Models, Long Prompts and Instruction Following 7
2.4 Self-Correction and Debugging 8
2.5 Research Gap 8

3 Experimental Setup **10**

3.1 Benchmark and Protocol 10
3.2 How One Task Is Generated and Scored 10
3.3 Models and Roles 12
3.4 Metrics 13
3.5 Experimental Design 13
3.6 Reproducing the Baseline 14
3.7 Reproducibility and Constraints 14

4 Method **17**
4.1 Problem Formulation . . . 17
4.2 Starting Point: Why No Memory Fails . . . 18
4.3 Final Method . . . 19

5 Results **24**
5.1 Main Comparison . . . 24
5.2 Learning Curve . . . 25
5.3 Why Text Memory Declines and Code Memory Improves . . . 26
5.4 Error Analysis . . . 27
5.5 Ablations: Checklist Only and Fixed Code Prefix . . . 28
5.6 Robustness to Task Order . . . 29

6 Discussion **34**
6.1 The Role of Memory Representation . . . 34
6.2 What Is Learned . . . 34
6.3 Cost . . . 35
6.4 Limitations and Threats to Validity . . . 35
6.5 Ethical Considerations and Impact . . . 37
6.6 Project Management . . . 37

7 Conclusion **39**
7.1 Answers to the Research Questions . . . 39
7.2 Future Work . . . 40

Bibliography **41**

List of Figures

3.1 One task in the pipeline. Black: the official WebGen-Bench setup (No Memory). Blue: the memory module, which reflects after each task and adds text to the prompt of later tasks only. 11

3.2 The first screenshot the tester received for task 000006 (seed-11 order). (a) An undeclared package stops the whole site. (b) The numbered dashed boxes are drawn by the tester to name the elements it can click; they are not part of the website. 12

3.3 The three memory designs. Each arrow names the failure that motivated the next design. Accuracy is on the manifest order (101 tasks, one run). . 14

4.1 The memory flow of the final method. 18

4.2 The prompt the final method gave the generator for task 000041 (seed-11 order), shortened. The requirement comes first and stays binding; the checklist repeats it item by item with lessons attached; the code pool adds verified fragments with the number of earlier tasks that failed in the same way. 20

5.1 Cumulative and rolling-10 accuracy by execution order for No Memory, text memory and whole-file code memory. [TODO: Planned: add the final version.] 26

5.2 Rolling 10-task mean of the per-task accuracy gap to No Memory, by execution order. Positive values mean Long Memory is better. 26

5.3 Code memory (whole-file) on task 000041, an online trading site (seed-11 order). Left: part of the code-pool entry in the prompt, a complete `App.vue` from an earlier food-donation site. Right: the generator returned the untouched Vite starter project, and all five test cases failed. The final method shows only verified lines of such a file, without the `<script>` and `<template>` tags (Figure 4.2). 31

5.4 Rolling 10-task mean of the per-task accuracy gap to the No Memory run of the same order, by execution order. Orange: whole-file code memory; blue: the final method with fragments only. Positive values mean memory is better. The whole-file dip after task 45 in seed 11 coincides with the scaffold outputs. 32

List of Tables

2.1 Closest related studies. Update: how the memory changes across tasks. Consumer: the model that uses the memory. Order: results compared across task orders. “no” and “–” mean not evaluated, not a shown limitation. 9

3.1 Model roles. The appearance grader is the only deviation from the paper, which uses GPT-4o. 12

3.2 Experimental design. Memory content is what is added to the generator’s prompt; the designs are those of Figure 3.3. 15

3.3 Placement of the self-hosted models on the two H200 GPUs (memory reserved by vLLM). 16

4.1 Constants of the final method. Several were revised in response to failure analysis on development runs, mainly seed 11 (Section 5.6.1); all were fixed before the final evaluation runs, and none was set by a systematic search. 23

5.1 Main results on all 101 tasks in the manifest order (single run per arm). Yes, Partial, No and Start failed are percentages of the 647 test cases; Accuracy = Yes $+0.5\times$ Partial. Appearance is the mean grade over all 101 tasks (1–5; tasks without a working site count as 0). Paired Δ is the mean per-task difference to No Memory with a 95% bootstrap confidence interval (CI); it can differ from the difference of pooled accuracies because tasks have different numbers of test cases. Zero: tasks where every test case fails. Best values in bold. 25

5.2 Prompt content by segment (tasks 1–30 / 31–60 / 61–101). 27

5.3 Generated sites with code problems (number of tasks; 1–30 / 31–60 / 61–101, total in brackets). 27

5.4 Ablation over the 101 tasks in the manifest order (single runs). Accuracy is pooled over checks; appearance averages tasks with a grade. 29
5.5 Task-order experiments: accuracy (%) of No Memory (NM), whole-file code memory (WF) and the final method with fragments (F), paired difference to No Memory with 95% bootstrap CI, and zero-score tasks. Single run of each arm per order. [TODO: Status: manifest (final method) running, expected 10-16; seed 47 running; interleaved queued. Fill in when complete.] 30

Chapter 1

Introduction

1.1 Background

1.1.1 Website Generation with Language Models

Agents built on Large Language Models (LLMs) can now turn a natural-language request into a running website [19]. Benchmarks such as WebGen-Bench start the generated project and let a Vision-Language Model (VLM) tester check each requested feature through the interface [19, 11], so a site that only looks plausible still fails.

The strongest results on such benchmarks come from large proprietary models. Small open models are attractive for a different reason: they can be run locally, at low cost and without sending data to a third party. WebGen-LM-7B, the 7-billion-parameter generator released with WebGen-Bench, was fine-tuned specifically for this task, yet it reaches only 28.4% accuracy under the official protocol [19]. Improving small generators without retraining them is therefore a practical problem as well as a research one.

1.1.2 Every Task Is Solved in Isolation

In WebGen-Bench, as in most code-generation benchmarks, every task is solved independently. The agent starts each task with the same weights and the same empty context. An agent that produced a blank page on one task because it imported a package that was not installed will make the same mistake on the next task, and on the one after that. In our reproduction of the baseline (Section 4.2), almost a third of the 101 tasks received a score of zero, and many of these failures had the same few causes: an undeclared dependency, a missing import, or a template that does not compile.

A human developer who has broken a build once avoids the same mistake the next time; these failures are exactly what experience should remove.

1.1.3 Long-Term Memory for Agents

Recent work on agent memory gives LLMs this ability: after each task the agent reflects on its trajectory, stores lessons or procedures, and retrieves them for later tasks [36, 31, 21, 32, 5]. Almost all of these systems use large models and store memory as natural-language insight. Small models, however, follow many instructions less reliably [10, 13, 17], are distracted by irrelevant context [24] and degrade as prompts grow [8, 18]. A memory that adds a paragraph of lessons to every prompt may help a large model and hurt a small one; which *form* of memory a small model can use is an open question.

1.2 Problem Statement

This dissertation asks whether a system built around a small website generator can get better with experience, without any change to the generator's weights. We use *long-term memory* in an operational sense: information is written after a task, persists across tasks, is retrieved under a stated condition, and changes the prompt of later tasks. *Learning* accordingly means a persistent, outcome-dependent change in the system's future behaviour, held in an external memory; the generator's parameters never change, and the memory is maintained by a separate, larger memory model (Chapter 3). We study the question on WebGen-Bench with WebGen-LM-7B. The generator, the agent scaffold (Bolt.diy) and the official evaluation protocol are kept fixed; the treatment changes only the generator's prompt, through an external memory pipeline that adds its own computation (Chapter 6). The memory is built only from the agent's own earlier tasks and their evaluation results; the hidden evaluation of the current task is never used, so the comparison with the no-memory baseline is fair.

Two properties make this setting a demanding test of long-term memory. First, tasks arrive in a sequence and the memory available to a task depends on which tasks came before it, so the result may depend on task order. Second, under the official protocol the generator has a single attempt per task and no repair loop, so the memory must prevent an error rather than help fix it afterwards.

1.3 Research Questions

RQ1. Can a cross-task long-term memory, built only from the agent's own earlier tasks, improve the functional accuracy of a small website generator under the official WebGen-Bench protocol?

RQ2. What form of memory does a small generator actually use: textual lessons, structured requirement checklists, or code examples?

RQ3. How does the benefit change as experience accumulates, and how sensitive is it to the order in which tasks are presented?

RQ1 asks whether there is an effect at all. *RQ2* asks what produces it, which matters for designing memory for small models in other domains. *RQ3* asks whether the effect behaves like learning from experience, rather than like a fixed prompt change that would help equally on every task, and whether it survives a change of task order. The fixed-prefix ablation (Section 5.5) addresses the first part directly.

1.4 Scope

The study is deliberately narrow so that its comparisons are clean.

- **One benchmark, one generator, one scaffold.** All experiments use the 101 tasks of WebGen-Bench, WebGen-LM-7B and Bolt.diy; larger generators are future work. Other agents such as OpenHands or Aider are not studied, because changing the agent also changes how memory reaches the model.
- **Official protocol, no repair.** Each website is generated once and evaluated as it is, as in the original benchmark. We do not add build checks or repair loops, which would make the results incomparable with the published baseline.
- **No retraining.** Memory only changes the prompt. Fine-tuning on stored experience is outside the scope.
- **Self-hosted models only.** Generation, evaluation and memory processing all run on a university GPU cluster; no commercial API is used.

1.5 Contributions

1. A fully self-hosted reproduction of the WebGen-Bench No Memory baseline for WebGen-LM-7B (28.5% vs 28.4% reported), together with a failure analysis showing that a large share of zero scores are blank pages caused by basic code errors (Chapter 3 and Section 4.2).

2. A long-term memory pipeline for website generation with three components, a requirement checklist, a code pool of fragments copied verbatim from the chain's own compiled websites, and memory hygiene, each motivated by a measured failure of the previous version (Chapter 4).

3. Evidence that, for this small generator, short code fragments reduced the targeted failures where textual warnings did not reliably do so, and that a text-dominated memory fell behind No Memory late in the sequence. On the original task order whole-file code memory raises accuracy from 28.5% to 40.0% (paired gain 11.5 points, 95% confidence interval (CI) 6.0–16.9), and a checklist-only ablation gains 3.4 points, suggesting that the stored cross-task content accounts for most of the improvement in this run (Chapter 5). [TODO: Status: the final method on the manifest order is running (expected 10-16). Replace this item with its result and describe whole-file code memory as the intermediate design.]

4. A task-order study of cross-task memory, with the two arms run as alternating pairs and compared by task-paired confidence intervals, which shows that a single memory entry can turn into a failure mode under a different order, and that the failure was not observed after whole-file examples were replaced with verified fragments (Sections 5.6 and 5.6.1).

1.6 Organisation of Thesis

Chapter 2 reviews related work and states the gap; Chapter 3 describes the benchmark, models and reproduced baseline; Chapter 4 presents the memory method and how it was developed; Chapter 5 reports the results; Chapter 6 discusses their meaning and limits; and Chapter 7 answers the research questions.

Chapter 2

Background and Related Work

This chapter covers website-generation benchmarks, memory for LLM agents, small models and added prompt text, and self-correction, and then states the research gap (Section 2.5).

2.1 Website Generation Benchmarks

Code-generation benchmarks have moved from single functions towards whole applications. A website is a demanding target: it combines several files, a build configuration, a user interface and interaction logic, and a request in natural language usually lists several pages and features at once. Evaluating such output by reading the code is unreliable, because a project can look complete and still fail to start, or start and render a blank page. Recent benchmarks therefore start the generated project and test it through its interface. Related benchmarks test agents that operate existing websites, such as Mind2Web and WebArena [6, 40], generate a single page from a screenshot and compare it visually, as in Design2Code [27], or change a code repository and run its tests, as in SWE-bench [14].

WebGen-Bench [19] contains 101 website requirements from 20 application domains, each paired with a set of functional test cases (647 in total). A generated project is installed and started, and a Graphical User Interface (GUI) agent based on WebVoyager [11] executes each test case and returns *Pass*, *Partial* or *Fail*. A separate VLM grades the appearance of the landing page from 1 to 5. The authors also release WebGen-LM, generators fine-tuned from Qwen2.5-Coder [12] on agent trajectories; with the Bolt.diy scaffold, WebGen-LM-7B reaches 28.4% accuracy and an appearance score of 2.5. Larger general models such as Qwen2.5-Coder-32B score lower (9.5%), which

makes the 7B model a good subject: it is neither at the floor nor at the ceiling of the benchmark.

WebGen-Bench evaluates generators inside agent scaffolds rather than as bare models. In the Bolt.diy scaffold used here, the model writes the project as a sequence of file actions in its reply, the scaffold applies them to a template project, installs dependencies and starts a development server. The official protocol gives the generator a single attempt: there is no build check and no repair loop, and a project that does not start receives zero for all of its test cases. This is important for our study, because it means memory cannot help by fixing errors after they happen; it can only change what the generator writes in the first place.

The benchmark reports single runs without error bars, citing computational cost, and defines no interaction between tasks [19]; with memory, task order becomes a treatment variable (Section 5.6).

2.2 Memory for LLM Agents

An agent memory has three parts: what is written after a task (raw trajectories, insights, procedures or code), how it is maintained (added, merged, updated or forgotten), and how it is retrieved and presented to the model for a new task. Most work varies the first two and assumes the consuming model can use whatever is retrieved. The paragraphs below follow the same division and end with the question that matters most here: which form of memory transfers to a new task.

Experience and insight memory. ExpeL [36] lets an agent collect trajectories, extract natural-language insights, and maintain them with add, edit, upvote and downvote operations. ReasoningBank [21] distils reusable reasoning strategies from both successful and failed trajectories. Generative Agents [23] rank memories by recency, importance and relevance. Reflexion [25] stores verbal self-reflections after a failed attempt, but reuses them on retries of the same task rather than on new tasks.

Skill and code memory. Voyager [31] stores executable skills as code and retrieves them by description; Agent Workflow Memory [32] stores reusable workflows. Metis [5] promotes recurring plans to validated tools that the agent executes; with a GPT-4o executor and memory built only from earlier tasks, its code memory reaches 53.3% task completion against 73.3% for text memory, brittle under task variation. Our fragments are different: they are placed in the prompt to be imitated, not executed.

Exemplars and evolving context. Synapse [38] abstracts earlier trajectories before using them as exemplars, Dynamic Cheatsheet [29] keeps strategies and code snippets updated at inference time, and ACE [35] edits a context playbook incrementally to avoid losing detail. Two lessons shaped our design: examples should be stripped of details from their source task, and entries should be versioned rather than rewritten.

Which form of memory transfers. Memory Transfer Learning [15] finds, with strong models, that abstract insights transfer best across domains and concrete trajectories can transfer negatively. Neither it nor Metis tests small models; our results suggest that the preferred form depends on the model that must use the memory.

Cross-task reuse on WebGen-Bench. Super Library Agent [28] also builds WebGen-Bench applications in sequence, sharing a library of components. It targets maintainability: on 24 single-topic tasks with a strong backbone, accuracy changes insignificantly (76.04% to 77.21%) while code size falls. It stores no lessons from failures and does not vary task order; we study correctness with a small generator on all 101 tasks, with order as a variable.

Memory maintenance. Mem0 [4] decides at write time whether a new memory should be added, should update an old one, or is redundant. Memento [39] learns the utility of stored cases from their outcomes. MemGPT [22] manages several tiers of memory for long interactions. Our hygiene component borrows these ideas: deduplication at storage time and a utility score for each memory.

Task order in continual agent evaluation. Benchmarks for agents that learn across tasks report that the order and composition of the task stream affect results. Evo-Memory [33] finds that baselines vary substantially between easy-to-hard and hard-to-easy sequences, and AgentCL [26] shows that memory methods can only be distinguished on streams that expose opportunities for reuse. We therefore compare orders in which tasks of the same domain are adjacent with orders in which they are mixed (Section 5.6).

2.3 Small Models, Long Prompts and Instruction Following

Several results suggest that adding text to a prompt is not free. Models become less likely to satisfy all instructions as their number grows [10, 13]; information in the middle of a

long context is used less than at its start or end [18]; irrelevant context distracts models [24]; and longer inputs alone can reduce performance even with perfect retrieval [8]. Models can be trained to prioritise instructions by source [30], which motivates keeping the user requirement explicitly above any memory content. Small models also struggle to benefit from long reasoning chains [17], which is consistent with our negative result for a short planning step (**??**).

These findings were obtained mostly on reasoning and question-answering tasks, but they point to a specific risk for memory. A memory that grows with experience adds more and more text to every prompt. For a large model this may be harmless; for a small one it may bury the user's requirement, introduce instructions that conflict with it, or present advice the model cannot turn into code. Whether a small generator can use memory at all, and in which form, therefore cannot be assumed from results obtained with large models.

2.4 Self-Correction and Debugging

Reflection on failed attempts is related to self-correction methods such as Self-Refine [20], Self-Debug [3] and Chain-of-Verification [7]. These methods repair the *same* task with feedback. We do not allow this in the formal comparison, because repairing a benchmark task using its hidden evaluation results would leak the answer. Reflection is used only to write memory for *later* tasks. The investigation steps we ask the reflection model to follow are modelled on systematic debugging [34].

2.5 Research Gap

Table 2.1 compares the closest studies along the dimensions that matter for this dissertation. Three gaps follow from it. First, cross-task memory has been studied almost only with large agents; whether a small generator benefits, and from which form of memory, remains underexplored. Second, on WebGen-Bench the only cross-task study targets maintainability with a strong backbone on single-topic suites, and reports no clear improvement in functional accuracy. Third, memory results are usually reported for a single task order, although the memory available to a task depends on the tasks before it. This dissertation addresses all three: it measures the effect of memory on functional accuracy for a 7B generator, retaining the official single-attempt procedure within each task and adding memory only between tasks; examines how textual, checklist and code-based memory behave across successive system versions (which differ in

more than representation); and repeats the comparison under different task orders with task-paired confidence intervals.

Table 2.1: Closest related studies. Update: how the memory changes across tasks. Consumer: the model that uses the memory. Order: results compared across task orders. "no" and "–" mean not evaluated, not a shown limitation.

Study	Memory form	Update	Consumer	Website tasks	Order
ExpeL [36]	textual insights	add, edit, up/down-vote	large	–	no
ReasoningBank [21]	reasoning strategies	append from successes and failures	large	–	no
Voyager [31]	executable skills	add verified skills	large	–	no
Metis [5]	text + executed code tools	promote recurring procedures	large	–	no
Memory Transfer Learning [15]	four representations	fixed (transfer study)	large	–	no
Super Library Agent [28]	shared component library	library grows with tasks	large	24 tasks	no
Evo-Memory [33]	benchmark of memory methods	varies	various	–	yes
This work	checklist + code fragments + text	dedup, utility retirement, versioned code	**7B**	**101 tasks**	**yes**

Chapter 3

Experimental Setup

3.1 Benchmark and Protocol

We use all 101 WebGen-Bench tasks. They come from 20 application domains of unequal size, from eight tasks each (company brochures, learning platforms, discussion forums, e-commerce) down to two (personal blogs, streaming, email clients). The benchmark's manifest order groups tasks into domain blocks; Section 5.6 adds shuffled orders. We follow the official protocol: the generator produces the project once, there is no repair loop, and a project is passed to evaluation as soon as its dependencies install and the development server starts. Test cases without an evaluation record count as failed, as in the official scoring script. An earlier phase of this project used a stricter build gate (`vue-tsc`) and a more generous repair loop; those results are not comparable with the paper and are not reported as main results.

Figure 3.1 shows one task in the evaluation pipeline. The memory module is the only component that differs between arms; the generator, the scaffold, the tester and the scoring are shared.

3.2 How One Task Is Generated and Scored

This section follows one task through the unmodified system, so that it is clear what the memory later changes. Task 000006 asks for a solar dashboard:

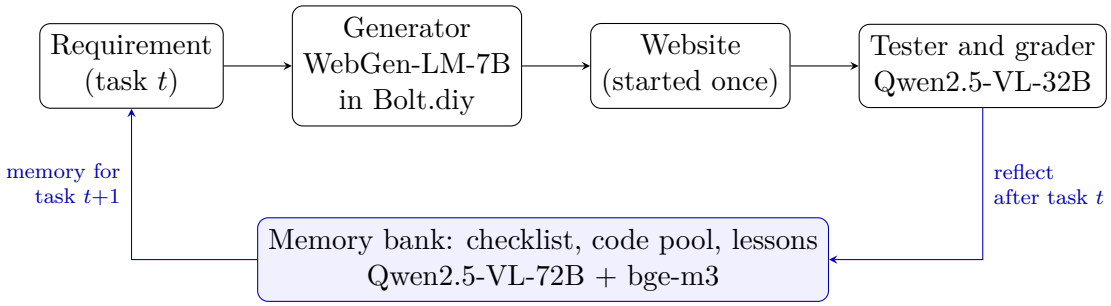


Figure 3.1: One task in the pipeline. Black: the official WebGen-Bench setup (No Memory). Blue: the memory module, which reflects after each task and adds text to the prompt of later tasks only.

```
Please implement a solar dashboard to display real-time and historical solar
data. The dashboard should have functionalities for real-time data display,
historical data query, and chart analysis. [...] Configure the background
    color
to honeydew, with components using dark olive green.
```

Generation. Under No Memory this text is the entire prompt. Bolt.diy is an open-source browser-based coding agent: it opens a starter project (for Vue, a Vite + Vue + TypeScript template whose `package.json` declares only `vue`), sends the requirement to the generator, and writes every file of the model's reply into the project. It then installs the dependencies and starts the development server. The generator has one reply; nothing in the official protocol inspects or repairs the code.

Functional testing. Each task comes with hidden test cases, on average six. For task 000006 the first two are "Verify the display of real-time solar power generation data" and "Test the display of real-time power consumption data", each with an expected result. A WebVoyager-style GUI agent [11] receives one test case at a time. At each step it sees a screenshot in which every interactive element carries a numbered box (Figure 3.2b), chooses an action such as clicking element 1 or typing into a field, and finally answers YES, PARTIAL or NO. The agent has a fixed step budget; if it runs out before reaching the target, the case counts as failed.

Appearance grading. A vision model grades a full-page screenshot from 1 to 5 for layout, colour and completeness. Both the tester and the grader are models acting as judges, which are known to have biases [37]; we therefore keep their raw transcripts and, in Section 6.4, check a sample of their verdicts by hand.

Figure 3.2 shows what the tester saw for this task in two runs. Under No Memory the model imported the chart library `vue-chartjs`, which the template does not declare; Vite cannot resolve the import, the page shows only the error overlay, and all test cases fail. With the final memory the generator built the three requested views without the undeclared package, and all test cases passed.



(a) No Memory: score 0.

(b) Code memory (fragments): score 1.0.

Figure 3.2: The first screenshot the tester received for task 000006 (seed-11 order). (a) An undeclared package stops the whole site. (b) The numbered dashed boxes are drawn by the tester to name the elements it can click; they are not part of the website.

3.3 Models and Roles

All models are self-hosted on the university CS ML cluster (two NVIDIA H200 GPUs per job, served with vLLM [16]). No commercial LLM service is called. Table 3.1 lists the roles.

Table 3.1: Model roles. The appearance grader is the only deviation from the paper, which uses GPT-4o.

Role	Model
Generator	WebGen-LM-7B (BF16), Bolt.diy scaffold, single generation
Functional tester	Qwen2.5-VL-32B-Instruct [1] (WebVoyager-style GUI agent)
Appearance grader	Qwen2.5-VL-32B-Instruct (paper: GPT-4o)
Memory / reflection	Qwen2.5-VL-72B-Instruct-AWQ
Embeddings	bge-m3 [2]

The generator. WebGen-LM is a family of website-generation models (7B, 14B and 32B) released with WebGen-Bench [19]. Each is Qwen2.5-Coder-Instruct [12] of the same size, fine-tuned with supervised learning only. The training data come from WebGen-Instruct, 6,667 website instructions decontaminated against the benchmark: the authors ran DeepSeek-V3 in Bolt.diy on 2,000 of them and kept the 600 trajectories whose websites scored at least 3 for appearance. WebGen-LM-7B therefore already knows the Bolt.diy output format, which makes it the natural small baseline for this benchmark: it is not a general model prompted to build websites, but a model distilled for exactly this scaffold.

The memory model is deliberately different from the evaluator, so that reflection cannot copy the evaluator's grading behaviour. vLLM applies the model repository's generation configuration by default; WebGen-LM-7B ships with a repetition penalty that Bolt.diy never sends, so we disable it for the generator to match the paper's setting.

3.4 Metrics

For each task with n test cases, of which p pass and q are partial,

$$\text{accuracy} = \frac{p + 0.5\,q}{n}. \tag{3.1}$$

We report accuracy over all test cases, the mean appearance score (1–5), and the number of *zero-score* tasks. Because test cases within a task are not independent, comparisons are paired by task rather than by test case: for two arms we report the mean per-task difference in accuracy with a 95% percentile bootstrap CI obtained by resampling tasks [9], together with the numbers of tasks won, lost and tied. This interval is conditional on the observed runs: it reflects which tasks happened to be in the benchmark, but not the variation between repeated runs of the same memory chain (about five points, Section 5.6.1), nor the dependence between tasks that a memory creates. We therefore say that an interval "excludes zero" rather than calling a gain significant. Accuracy pools test cases (micro); the learning curves average tasks (macro).

3.5 Experimental Design

We compare three memory designs, shown in Figure 3.3 and described in Chapter 4. *Text memory* stores lessons written in natural language and attaches them to a requirement checklist. *Code memory (whole-file)* keeps the checklist and adds a code pool of examples

taken from the chain's own working websites; some of these examples are complete files. *Code memory (fragments)*, the final method, shows only short verified fragments. The method was refined over several iterations; only these three designs were run on all 101 tasks, and only they are reported.[1]

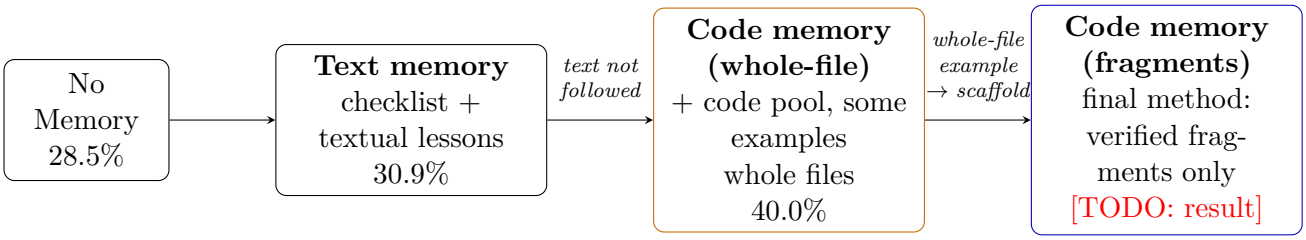


Figure 3.3: The three memory designs. Each arrow names the failure that motivated the next design. Accuracy is on the manifest order (101 tasks, one run).

Table 3.2 lists every arm, the task orders in which it was run and the question it answers. All arms use WebGen-LM-7B with Bolt.diy under the official protocol, cover all 101 tasks and start from an empty memory; each arm is run once per order. The task orders are defined in Section 5.6.

Memory never uses information from the current task itself (leave-one-task-out), and evaluation results are never fed back into the same task.

3.6 Reproducing the Baseline

No Memory on all 101 tasks reaches 28.5% accuracy (paper: 28.4%) with a mean appearance score of 2.35 (paper: 2.5 with GPT-4o) and 30 zero-score tasks. The near-identical aggregate accuracy is evidence that the self-hosted pipeline approximately reproduces the published baseline; the appearance grader differs, and task-level agreement cannot be measured because the paper's per-task outputs are not available.

3.7 Reproducibility and Constraints

Every run starts from an empty memory and stores its configuration file, task order, prompts, retrieved and injected memory, raw evaluator transcripts and scores; the key files of every run are archived in the project repository. The generator is decoded deterministically, so differences between arms come from the prompt rather than from sampling, although the GUI tester is not deterministic.

[1] In the code repository these runs are labelled v4, v7 and v9.1.

Table 3.2: Experimental design. Memory content is what is added to the generator's prompt; the designs are those of Figure 3.3.

Arm	Memory content	Task orders	Question
No Memory	none (requirement only)	manifest, seeds 11, 29, 47, interleaved	Does the baseline reproduce the paper, and how much does it vary between runs?
Text memory	checklist with textual lessons	manifest	Does textual memory help a small generator?
Checklist only	checklist; memory neither read nor written	manifest	How much does decomposing the requirement add?
Code memory (whole-file)	checklist, lessons and code examples, some complete files	manifest, seeds 11, 29	Does code memory help, and does it survive a change of order?
Fixed code prefix	checklist and the final code memories from task 1; no memory	manifest	Is the gain learning, or just code in the prompt?
Code memory (fragments)	as above, but code examples are verified fragments only	manifest, seeds 11, 29, 47, interleaved	Does the final method hold across task orders?

Hardware. All experiments ran on one node of the School of Computer Science CS ML cluster (`foscsmlprd03`, seven NVIDIA H200 GPUs with 141 GB of memory each, Slurm scheduler), with at most two GPUs per user. A single long-running job held two GPUs and served every model with vLLM; all arms and task orders shared these servers (Table 3.3). Two tasks were processed in parallel, while evaluation was serialised by a lock, because the evaluator restarts the website process manager. From the first job on 5 October 2026 to the end of the 7B runs, the project used about [TODO: update] H200 GPU-hours.

Code and data availability. The code, configuration files and the archived run records (prompts, injected memory, scores and memory databases of every run) are publicly available at https://github.com/qiaopan/WebGen-Bench; the run records are in `archive/phase2-webgen-2026-10/`.

The main constraint is compute. A task takes about twelve minutes with memory and six without, so one 101-task run takes about a day on our two GPUs. As in the original benchmark, each configuration is therefore run once per task order. We compensate by

Table 3.3: Placement of the self-hosted models on the two H200 GPUs (memory reserved by vLLM).

GPU	Model (role)	Memory (GB)
GPU 0	Qwen2.5-VL-32B-Instruct, BF16 (evaluator)	83
	WebGen-LM-7B, BF16 (generator)	26
GPU 1	Qwen2.5-VL-72B-Instruct-AWQ (memory)	113
	bge-m3 (embeddings)	2

pairing comparisons by task, by repeating the No Memory baseline in three orders to measure its run-to-run variation, and by repeating the memory method across orders rather than within one.

Chapter 4

Method

4.1 Problem Formulation

A run processes the tasks $t_1, \ldots, t_n$ ($n = 101$) in a fixed order. Task t_i has a requirement r_i and hidden test cases $c_{i,1}, \ldots, c_{i,m_i}$. A generator G with fixed weights maps a prompt x_i to a project $y_i = G(x_i)$, which the evaluator E scores as $a_i = E(y_i, c_{i,\cdot}) \in [0, 1]$ with the accuracy defined in Chapter 3. Under No Memory, $x_i = r_i$. With memory, the prompt is

$$x_i = r_i \oplus f(r_i, M_{i-1}), \qquad M_i = u\big(M_{i-1}, r_i, y_i, \tau_i, a_i\big), \tag{4.1}$$

where $M_0 = \emptyset$, f selects and formats memory for the current requirement, $\oplus$ appends text after the requirement, τ_i is the evaluator transcript, and u reflects on the finished task and updates the memory. Two constraints make the comparison fair: f only reads M_{i-1}, which is built from earlier tasks, so the hidden evaluation of t_i never reaches its own prompt; and G, E and the protocol are identical across arms. The question is whether the mean of a_i is higher with memory than with $x_i = r_i$, and which design of f and u achieves it. Algorithm 1 gives the final design, which Section 4.3 specifies in full; Section 4.2 and **??** first explain the failures that motivated each component.

Figure 4.1 shows the same loop as a diagram. StaticAnalyse compiles every component of a site with `@vue/compiler-sfc` and records undeclared package imports, Vue functions used without import, and compile errors. UpdatePool activates an entry for each recorded problem whose trigger holds (two tasks, or one for a whole-site failure) and, for each new or changed entry, takes the key lines that the memory model picks from an earlier successful site and accepts them only if they are verbatim, at most 15 lines, free of `<script>`/`<template>` tag lines, and include the needed import and a line that uses it; after two failed picks a rule-based extraction is used. A new version of an

Algorithm 1 Cross-task memory for one run (final version).

$S_0 = (M_0, F_0, C_0) \leftarrow (\emptyset, \emptyset, \emptyset)$ ▷ lessons, code facts, versioned code pool
for $i = 1, \ldots, n$ **do**
  $L \leftarrow \textsc{Checklist}(r_i)$ with at most three hard-match lessons from M_{i-1}
  $P \leftarrow \textsc{SelectActive}(C_{i-1}, r_i)$ ▷ ≤ 3 active entries
  $x_i \leftarrow r_i \oplus L \oplus P$; $y_i \leftarrow G(x_i)$; $(a_i, \tau_i) \leftarrow E(y_i)$
  $M_i \leftarrow \textsc{Reflect}(M_{i-1}, r_i, y_i, \tau_i, a_i)$ ▷ deduplicate; update utilities
  $F_i \leftarrow F_{i-1} \cup \textsc{StaticAnalyse}(y_i)$
  $C_i \leftarrow \textsc{UpdatePool}(C_{i-1}, F_i, \{y_j : j \leq i,\ a_j \geq 0.5\})$ ▷ activate, verify, version
end for

entry is added only when its example covers strictly more of the needed functions; the old version is kept but no longer injected. All three parts of the state persist in the run's database. Section 4.3.2 gives the details and Table 4.1 the constants.

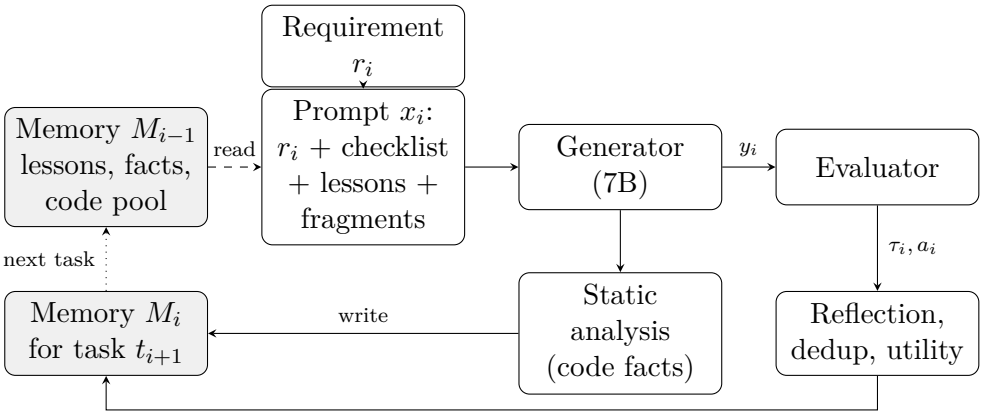


Figure 4.1: The memory flow of the final method. Dashed arrows read the memory built from earlier tasks; the lower row writes memory after evaluation. r_i: requirement; y_i: generated site; τ_i: evaluator transcript; a_i: score; M_{i-1} and M_i: the same memory store before and after task t_i. The evaluation of task t_i only reaches the memory used by later tasks, never its own prompt.

4.2 Starting Point: Why No Memory Fails

We began by inspecting the code that No Memory actually produced, rather than its scores. Two findings shaped everything that followed.

Blank pages from basic errors. Bolt.diy's Vue template declares only `vue` as a dependency, and WebGen-LM-7B never edits `package.json` (0 of 81 inspected responses).

When it imports `vue-router` or uses a Vue function such as `ref` without importing it, the whole site renders as a blank page and every test case fails. In an early Long Memory run, 26 of 74 tasks were blank, 19 of them because of an undeclared `vue-router` import. Many zero-score tasks therefore fail before their functional difficulty can be assessed: the site does not render because of a basic error that the model repeats.

Text is not followed. In a targeted diagnostic, not a powered experiment, we took nine tasks that had produced blank pages, kept their original prompts, and added a single sentence, phrased in several ways ("add vue-router to package.json", "do not use vue-router", "put pages in App.vue with `ref` and `v-if`", or the same sentence at both the start and the end). None recovered more than one task in three. Each phrasing was tried on three of the nine tasks; a task counted as recovered when the generated code imported no undeclared package and compiled. A ten-line code example of the same idea, tried on all nine, recovered seven. Because the text variants and the code example were tried on different subsets, this is indicative rather than conclusive; it suggested that memory might work better as code, in line with reports that small models follow instructions less reliably [10, 17].

4.3 Final Method

For each task, in order:

1. **Requirement checklist.** The memory model splits the requirement into items (Section 4.3.1).

2. **Retrieve lessons.** The memory model also splits the requirement into at most three functional *aspects* (a feature and the user action that triggers it) and appearance aspects; these are the retrieval queries. Functional lessons are retrieved by similarity and judged for applicability by the memory model as a hard match (same user trigger and same state change), a soft transfer (same kind of interaction) or a reject; the query is the aspect written as feature, user trigger, expected state change and a domain-free interaction pattern, and it is compared with the stored case (trigger, observation, cause, preventive action); only hard matches are attached to checklist items. A small common pool adds site-wide lessons that contain a concrete code construct.

3. **Code pool.** At most three verified fragments whose trigger condition holds (Section 4.3.2).

4. **Generate and evaluate.** One generation by WebGen-LM-7B; official functional and appearance evaluation.

5. **Reflect and store.** The memory model reflects on the evaluated task; new lessons are deduplicated against existing ones, utilities are updated, and code facts are computed for the code pool.

The products of step 5 are read by steps 2 and 3 of later tasks; this is the only path through which tasks influence each other. Figure 4.2 shows the resulting prompt for one task; under No Memory the prompt is the first paragraph only.

```
Please implement an online trading website for bidding and trading. [...] Set the
background color to lemon chiffon and the component color to chocolate.

FEATURE CHECKLIST (the requirement above, item by item; build every item):
1. Please implement an online trading website for bidding and trading.
3. allowing users to browse trading opportunities,
   Note from earlier websites: For the trading opportunities: use `v-for` to
   display the list with current status.
[... items 2, 4-8 ...]

CODE FROM EARLIER WEBSITES THAT WORKED:
Component fails: ref() not imported. Earlier websites broke here (a component
calls ref() without importing it from 'vue' (that page fails when opened),
19 tasks). Code that works:
    import { ref, onMounted } from 'vue'
    const wheelRef = ref<HTMLElement | null>(null)
Vue compile error: v-model on prop. [...] (4 tasks). Code that works:
    <SearchBar :modelValue="searchQuery"
               @update:modelValue="emit('update:searchQuery', $event)" />
[... one more entry ...]

Build every checklist item; the notes only warn about mistakes earlier websites made.
```

Figure 4.2: The prompt the final method gave the generator for task 000041 (seed-11 order), shortened. The requirement comes first and stays binding; the checklist repeats it item by item with lessons attached; the code pool adds verified fragments with the number of earlier tasks that failed in the same way.

4.3.1 Requirement Checklist

The original requirement remains at the top of the prompt and is declared binding. The checklist below it repeats every requested feature, so that memory content cannot crowd a feature out of the model's attention [18, 30]. Lessons are attached to the item they concern rather than listed separately, which avoids a separate list being read as the full scope of the task.

4.3.2 Code Pool

The code pool is computed without an LLM for detection:

- **Problem facts.** After each task, code analysis of the generated site records undeclared imports (`import:<package>`: an import statement whose package `package.json` does not declare), Vue functions used without import (`vue-api`: a Vue function name followed by `(` or `<` in a script that neither imports nor defines it), both found by pattern matching on the source, and compile errors from compiling each Single-File Component (Vue) (SFC)'s script and template together with `@vue/compiler-sfc`, as Vite's Vue plugin does (without type checking or bundling).

- **Trigger.** A problem key becomes active after it has appeared in two different tasks, following Metis's rule of promoting a procedure only after it recurs [5]. Whole-site failures that code detects objectively (an undeclared package, a compile error) become active after one occurrence. Missing-import and compile-error entries apply to every Vue site; other entries are applied only if the memory model judges their condition to hold for the requirement.

- **Source.** The example comes from the chain's own earlier websites that scored at least 0.5, compile and have no undeclared import, preferring the highest-scoring site. The threshold of 0.5 is a design choice (at least half the checks passed), not a tuned value. Successful sites supply *how* to write it; failed sites decide *when* it is needed [21].

- **Fragment selection.** The memory model picks the key lines; code then verifies that the lines are verbatim from the source file, contain at most 15 lines, contain no `<script>`/`<template>` tag lines, and include the import of the most frequently missed function plus another line that calls it (the same pattern: its name followed by `(` or `<`). If the model fails twice, a rule-based extraction is used. An entry is updated by adding a new version only when the new example covers strictly more of the needed functions; the old version is kept for provenance [35].

- **Slots.** At most three entries per task, at most one of them for a missing package.

Verified in this dissertation means only that a fragment is verbatim from a source file that compiles and comes from a site with score ≥ 0.5; it does not mean that the fragment is correct for the new task. Because the triggers are general, the code pool behaves partly like a learned prompt prefix: once active, many later tasks receive the same fragment.

The fixed-prefix ablation (Section 5.5) separates this from learning. Table 4.1 lists the constants of the final configuration.

4.3.3 Memory Hygiene

- **Concrete only.** Common-pool entries without a code construct (for example `@click`, `v-if`, an `import` line) are dropped. Rewriting may generalise the *condition* under which a lesson applies but not the *action*.
- **Deduplication at storage time.** A new lesson is compared with similar stored lessons; if the memory model judges it to share the same root cause and fix, it is recorded as support for the old lesson instead of a new entry [4].
- **Utility and retirement.** For each lesson we count injections u and recurrences r (the same problem reappearing after injection), and compute utility $(u - r + 1)/(u + 2)$ [39, 36]. Ranking uses support $\times$ severity $\times$ utility; a lesson injected at least five times with utility below 0.3 is no longer injected. Utility is a heuristic: a task after injection that does not repeat the problem may not have needed the lesson, and a lesson used on hard tasks can be penalised. The constants (two occurrences, five injections, 0.3) are design choices that were not tuned.
- **No hand-written conclusions.** Instructions to the reflection model describe how to investigate (follow the code path, trace styles from element to global stylesheet, list candidate causes and rule them out) but never what the answer is.

Table 4.1: Constants of the final method. Several were revised in response to failure analysis on development runs, mainly seed 11 (Section 5.6.1); all were fixed before the final evaluation runs, and none was set by a systematic search.

Parameter	Value
Lesson retrieval	bge-m3 embeddings; 5 candidates per requirement aspect (Section 4.3), at most 3 functional aspects; similarity ≥ 0.35; only hard matches kept
Lessons injected	at most 3, total length at most $0.5\times$ the requirement
Common pool	6 candidates, at most 3 kept; only entries with a code construct
Deduplication	candidates with similarity ≥ 0.55, then same root cause and fix judged by the memory model
Retirement	at least 5 injections and utility < 0.3
Code-pool trigger	2 tasks; 1 task for whole-site failures
Code-pool source	earlier site with score ≥ 0.5 that compiles
Fragment	verbatim, at most 15 lines, no tag lines
Code-pool slots	at most 3 per task, at most 1 for a missing package
Memory model	Qwen2.5-VL-72B-Instruct-AWQ, at most 4096 output tokens
Leakage control	a task never reads memory written from itself

Chapter 5

Results

Each arm is a single run over all 101 tasks. Sections 5.1 to 5.5 use the manifest order; Section 5.6 repeats the comparison in other orders. Accuracy pools all test cases (as in the official script); the learning curves average per-task accuracy, so their end points differ slightly.

5.1 Main Comparison

Table 5.1 reports the main comparison in the format of the WebGen-Bench results table [19], with the published WebGen-LM-7B result as the first row. Every arm uses the same generator, scaffold and evaluators; the treatment is the prompt supplied by the external memory pipeline, including the checklist and any cross-task content.

Whole-file code memory raises the share of passed test cases from 23.8% to 34.5% and halves the number of zero-score tasks; per task it beats No Memory on 54 tasks, loses on 24 and ties on 23. The gain over text memory is +8.9 points ($[2.9, 15.1]$; 53 wins, 29 losses, 19 ties). For neither text memory nor the checklist alone does the interval exclude zero. For scale, No Memory has no cross-task state, so changing the order should not change its prompts; its three runs under different orders (28.5%, 27.9% and 28.6%, Section 5.6) give an approximate estimate of rerun variability, although they are not repeats of one order: its run-to-run spread of 0.7 points is far smaller than the 11.5-point gain of code memory. A Long Memory chain varies more than No Memory, because its memory diverges from the first task (Section 5.6.1). Appearance in Table 5.1 counts tasks without a working site as 0, so it rises when more sites work; on the 68 tasks where both No Memory and whole-file code memory produced a graded site, the means are 3.25 and 3.22. [TODO: Planned (existing data): split the gain into (a) tasks

Table 5.1: Main results on all 101 tasks in the manifest order (single run per arm). Yes, Partial, No and Start failed are percentages of the 647 test cases; Accuracy = Yes $+ 0.5\times$ Partial. Appearance is the mean grade over all 101 tasks (1–5; tasks without a working site count as 0). Paired Δ is the mean per-task difference to No Memory with a 95% bootstrap CI; it can differ from the difference of pooled accuracies because tasks have different numbers of test cases. Zero: tasks where every test case fails. Best values in bold.

Arm	Yes	Partial	No	Start fail.	Acc.	Appear.	Zero	Paired Δ [95% CI]
WebGen-LM-7B (paper)[a]	24.9	7.1	68.0	0.0	28.4	2.5	–	–
No Memory (ours)	23.8	9.4	66.5	0.3	28.5	2.35	30	–
Text memory	25.2	11.4	62.6	0.8	30.9	2.57	20	+2.6 [−3.6, 8.7]
Checklist only	26.4	10.7	61.4	1.5	31.8	2.47	18	+3.4 [−2.2, 8.7]
Code memory (whole-file)	**34.5**	11.1	**53.2**	1.2	**40.0**	**2.83**	**13**	**+11.5** [6.0, 16.9]
Code memory (fragments)	[TODO: running]							

[a] Reported by Lu et al. [19]; appearance graded by GPT-4o, ours by Qwen2.5-VL-32B, so the two are not directly comparable.

where only the memory arm produced a working site and (b) tasks where both arms worked, and report the paired gain and CI on (b). Do this for the final version once its manifest run is scored.]

5.2 Learning Curve

Figure 5.1 plots cumulative and rolling accuracy in execution order. Text memory and whole-file code memory are similar on the first 30 tasks (39.3% and 39.0%, versus 26.0% for No Memory), when the code pool has few active entries. On the remaining 71 tasks text memory falls below No Memory (27.8% vs 29.5%, 19 zero-score tasks each), while code memory keeps its level (40.4%, 9 zero-score tasks). On the last 35 tasks the gap is 38.3% (code) vs 26.0% (No Memory) vs 23.2% (text).

Figure 5.2 shows the same data as a paired gap to No Memory. Both memories lead early; between tasks 40 and 60 text memory falls up to 27 points below No Memory while code memory stays near or above zero, and from task 60 onwards code memory stays clearly positive.

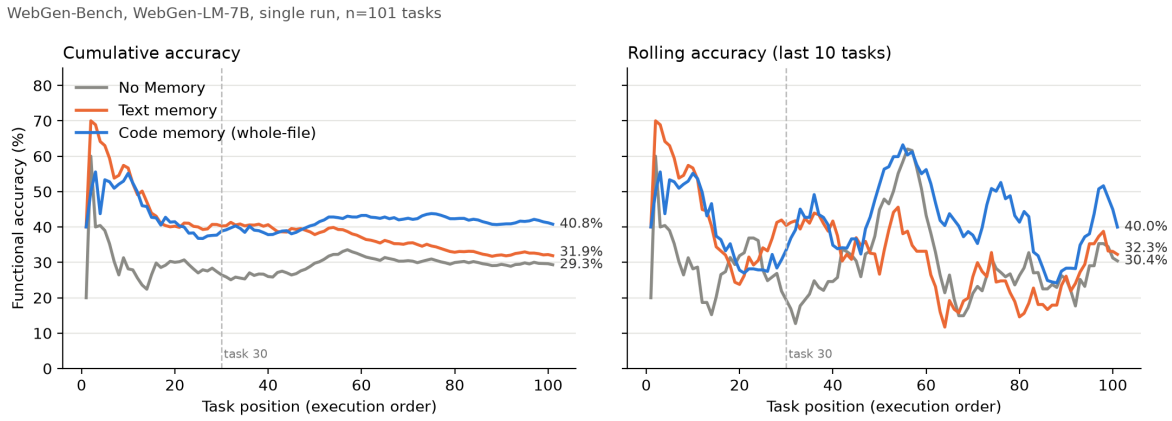


Figure 5.1: Cumulative and rolling-10 accuracy by execution order for No Memory, text memory and whole-file code memory. [TODO: Planned: add the final version.]

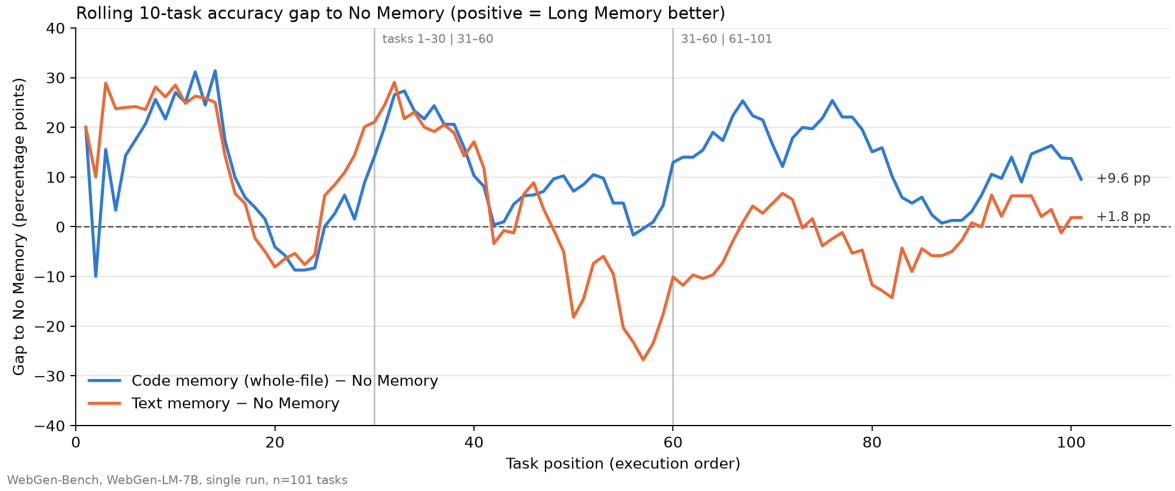


Figure 5.2: Rolling 10-task mean of the per-task accuracy gap to No Memory, by execution order. Positive values mean Long Memory is better.

5.3 Why Text Memory Declines and Code Memory Improves

Table 5.2 counts what reached the generator's prompt, and Table 5.3 counts code problems in the generated sites, obtained by compiling every site with `@vue/compiler-sfc` 3.5.43. A lesson is counted as *without a code construct* if it contains none of `@click`, `v-if`, `v-for`, `v-model`, `ref`, an `import` or inline code. In text memory such lessons appear in most tasks throughout and a separate "General notes" block becomes more

frequent over time, while nothing addresses blank pages. In code memory these lessons are rarer and the code pool supplies a fragment in almost every task after it activates.

Table 5.2: Prompt content by segment (tasks 1–30 / 31–60 / 61–101).

Prompt content	Text	Code (whole-file)
Tasks with a lesson without a code construct	23 / 25 / 28	12 / 4 / 11
Tasks with a “General notes” block	13 / 16 / 27	3 / 0 / 8
Tasks with a code-pool fragment	0 / 0 / 0	25 / 30 / 41
Lessons attached to checklist items	56 / 53 / 81	32 / 16 / 41
of which without a code construct (all tasks)	102 of 190	31 of 89

Table 5.3: Generated sites with code problems (number of tasks; 1–30 / 31–60 / 61–101, total in brackets).

Problem	No Memory	Text	Code (whole-file)
Whole-site break[a]	11 / 5 / 10 (26)	2 / 5 / 12 (19)	4 / 1 / 3 (**8**)
Missing Vue import on a page	7 / 7 / 9 (23)	14 / 10 / 7 (31)	12 / 4 / 6 (22)

[a] An undeclared package import or a Vue compile error.

These counts are correlational. Disentangling the contribution of each component requires the ablations in Section 5.5.

5.4 Error Analysis

Whole-site breaks fall from 26 tasks (No Memory) to 8 (code memory), whereas pages with a missing Vue import are about as frequent as without memory (23 vs 22): the code pool prevents the error that blanks the whole site more reliably than the per-page one, which remains a basic error of the 7B generator. Counted per file, 27 of 592 generated Vue components with code memory (4.6%) call a Vue function without importing it, against 27 of 586 (4.6%) for No Memory. The omission occurs in `App.vue` and in child components alike, and is unrelated to whether a component uses `defineProps`/`defineEmits`; in the same file other imports are usually correct. The two arms show the same per-file rate, which is consistent with a slip rather than a misconception that an example could correct, though we have not shown this directly; a site with more components is more likely to contain one.

This slip was not reduced by a better example: a variant that showed both `ref` and `computed` removed it in one of four affected tasks. As the official protocol forbids repairing code after generation, we report it as a limitation of the generator. Ten of code memory's 13 zero-score tasks are also zero for No Memory. [TODO: Planned (existing data): list the cause of each remaining zero-score task of the final version (blank page, missing feature, budget exhausted) instead of calling them hard.]

Appearance. Restricted to the 68 tasks where both arms produced a working site with an appearance grade, the mean appearance score is 3.25 for No Memory and 3.22 for whole-file code memory. We observed no meaningful difference on this common subset; because memory changes which tasks enter the subset, this does not show that memory has no effect on appearance. The three appearance figures in this dissertation use different denominators: 2.35 averages all 101 tasks with failures as 0 (Table 5.1), 3.25 averages tasks with a grade (Table 5.4), and 3.25 vs 3.22 is the paired subset.

5.5 Ablations: Checklist Only and Fixed Code Prefix

Two complementary ablations use the manifest order: a checklist-only control and a retrospective fixed-code control using the code entries learned by the final method:

- **Checklist only:** the code-memory configuration with memory reads and writes disabled; each task sees only the requirement checklist. Checklist only − No Memory estimates the effect of requirement decomposition.
- **Fixed code prefix:** checklist only, plus the code-pool entries that the final method had learned by the end of its manifest run (Section 4.3.2), injected from the first task with the same selection rule, still without memory. The code is not written by hand; it is the method's own learned content. Because it is taken from the end of a complete run, this control has access to experience that an online learner only gains later; it is a retrospective, oracle-style comparison rather than an online one. Fixed prefix − checklist only estimates the effect of having this code in the prompt, and final method − fixed prefix the effect of accumulating experience and selecting it per task.

If the fixed-code control performs similarly to the final method, the contribution is better described as automatically discovering effective prompt code from the agent's

own failures and successes than as continual accumulation. The comparison does not isolate accumulation alone, because the two conditions also differ in textual lessons and memory maintenance.

Table 5.4 reports the checklist-only ablation over all 101 tasks; the fixed-prefix run is queued after the seed-47 order (expected about 10-18).

Table 5.4: Ablation over the 101 tasks in the manifest order (single runs). Accuracy is pooled over checks; appearance averages tasks with a grade.

	Acc. 1–30	Acc. 31–101	Acc. all	Zero-score	Appearance
No Memory	26.0	29.5	28.5	30	3.25
Checklist only	33.1	31.3	31.8	18	3.23
Fixed code prefix			[TODO: running]		
Code memory (whole-file)	39.0	40.4	40.0	13	3.21
Code memory (fragments)			[TODO: running]		

Paired by task (a mean of per-task differences, so it can differ slightly from the difference of the pooled accuracies in Table 5.4), checklist only − No Memory is +3.4 points (bootstrap 95% CI $[-2.2, 8.7]$; 51 wins, 27 losses, 23 ties), and whole-file code memory − checklist only is +8.1 ($[2.9, 13.2]$; 46/28/27). Requirement decomposition therefore mainly prevents whole-site failures (zero-score tasks fall from 30 to 18) but does not reliably raise accuracy, and its advantage shrinks from +7.1 points in the first 30 tasks to +1.8 afterwards. Memory contributes the larger share, with an interval that excludes zero, and its advantage over the checklist grows from +5.9 to +9.1 points between the two segments, consistent with the code pool becoming active only after a problem has recurred.

[TODO: Status: fixed-prefix run queued after seed 47. Fill in its row and the differences fixed prefix − checklist only and final − fixed prefix. Wording depends on the result: if fixed prefix ≈ final, describe the contribution as automatically discovering effective prompt code; if the final method is clearly better, as accumulation and per-task selection.]

5.6 Robustness to Task Order

With memory, the order of tasks determines which experience is available, so a single order is not sufficient evidence. In particular, in the manifest order tasks of the same

domain are adjacent, so a later task can always draw on a just-solved, similar website; a gain could therefore reflect a curriculum effect rather than general learning.

We therefore compare two kinds of order, each run twice, every run starting from an empty memory:

Domains adjacent. The order of the 20 domain blocks and the order of tasks within each block are shuffled, with seed 11 and seed 47. The manifest order, used for the main results, also belongs to this kind.

Domains mixed. A uniformly random order of all 101 tasks (seed 29), and an interleaved order that takes one task from each domain in turn, so that consecutive tasks always come from different domains.

In the shuffled and interleaved orders, Long Memory and No Memory are run as pairs on each task, alternating which arm runs first (ABBA), to reduce confounding from infrastructure drift and evaluator load. The manifest-order arms were run at different times and are reported separately. No Memory should not depend on order; its runs instead measure run-to-run noise (three completed, two running). The checklist-only ablation does not use memory and is run once. Results are task-level paired differences with bootstrap CIs.

Table 5.5: Task-order experiments: accuracy (%) of No Memory (NM), whole-file code memory (WF) and the final method with fragments (F), paired difference to No Memory with 95% bootstrap CI, and zero-score tasks. Single run of each arm per order. [TODO: Status: manifest (final method) running, expected 10-16; seed 47 running; interleaved queued. Fill in when complete.]

Kind	Order	NM	WF	F	F − NM	Zero NM / WF / F
Adjacent	Manifest[a]	28.5	40.0	–	–	30 / 13 / –
Adjacent	Shuffled blocks (seed 11)	27.9	33.1	35.4	+7.4 [1.6, 13.1]	29 / 21 / 16
Adjacent	Shuffled blocks (seed 47)	–	–	–	–	–
Mixed	Random (seed 29)	28.6	32.6	33.7	+5.5 [−0.5, 11.6]	29 / 21 / 18
Mixed	Interleaved	–	–	–	–	–

WF − NM: manifest +11.5 [6.0, 16.9]; seed 11 +4.4 [−1.9, 10.6]; seed 29 +4.5 [−2.2, 11.3].

[a] Not run as alternating pairs; the two arms were run at different times.

Development and validation. Seed 11 was used to develop the fragment fix (Section 5.6.1) and is therefore development data for the final method. Its manifest, seed-29,

seed-47 and interleaved runs started after it was frozen. All orders use the same 101 tasks, on which the method was developed, so this is a within-benchmark study with cross-order validation, not a held-out evaluation (Section 6.4).

The first two shuffled orders give almost the same result (Table 5.5): Long Memory leads by about 4.5 points, with zero-score tasks reduced from 29 to 21, but neither confidence interval excludes zero, and the gain is far smaller than in the manifest order. No Memory is stable across its three runs (27.9–28.6%), so the change is on the Long Memory side. Mixing domains does not remove the gain: the random order (seed 29) matches the shuffled-blocks order (seed 11), and from task 31 onwards Long Memory leads by 6.6 points in the random order (33.9% vs 27.3%). The gain is therefore not restricted to orders with adjacent same-domain tasks, although these results do not establish how much domain adjacency contributes.

Both runs lose much of their gain to the same failure (Figure 5.4). Long Memory returned the unmodified Vite scaffold on 19 tasks in seed 11 and 16 in seed 29, against none for No Memory in either order and two in the manifest order. Every scaffold occurred after the missing-package code memory, whose example was a complete `App.vue` of another site, began to be injected (task 36 in seed 11, task 19 in seed 29). Figure 5.3 shows one such case; Section 5.6.1 describes the fix.

```
CODE FROM EARLIER WEBSITES THAT WORKED:
Missing package: imports a package that
package.json does not declare. [...] Code
that works:
<script setup lang="ts">
import { ref } from 'vue'
import FoodDonationPage from
  './pages/FoodDonationPage.vue'
[... 3 more page imports ...]
const activeTab = ref('donation')
</script>
<template>
  <div class="app-container">
    <header>
      <h1>Food Distribution System</h1>
      [... 30 lines: nav, four pages ...]
</template>
```



Figure 5.3: Code memory (whole-file) on task 000041, an online trading site (seed-11 order). Left: part of the code-pool entry in the prompt, a complete `App.vue` from an earlier food-donation site. Right: the generator returned the untouched Vite starter project, and all five test cases failed. The final method shows only verified lines of such a file, without the `<script>` and `<template>` tags (Figure 4.2).

With the failure fixed (Section 5.6.1), no scaffold occurred in either order (19 → 0 and 16 → 0) and the gain over No Memory rose to 7.4 points in seed 11 (95% CI [1.6,

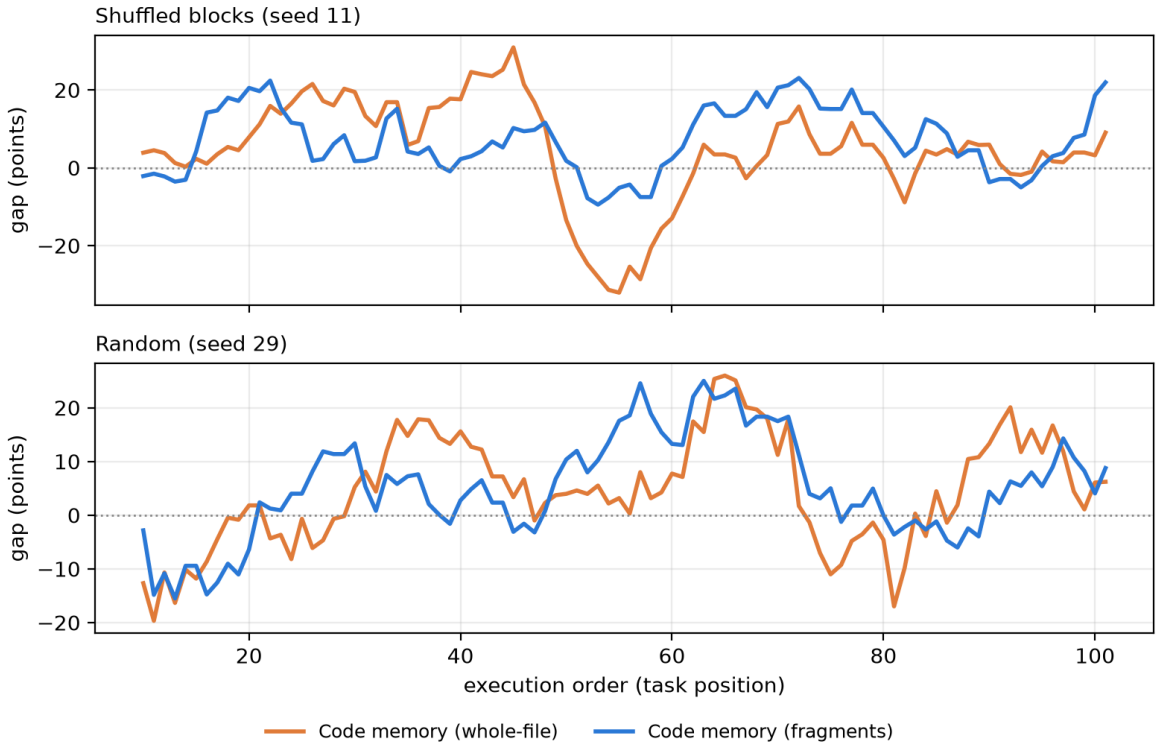


Figure 5.4: Rolling 10-task mean of the per-task accuracy gap to the No Memory run of the same order, by execution order. Orange: whole-file code memory; blue: the final method with fragments only. Positive values mean memory is better. The whole-file dip after task 45 in seed 11 coincides with the scaffold outputs.

13.1]) and 5.5 points in seed 29 ([−0.5, 11.6]), with zero-score tasks reduced from 29 to 16 and 18 (Table 5.5 and Figure 5.4). Positive gains were observed with domains adjacent and with domains mixed, although the interval for the mixed order includes zero; on this evidence the gain does not rely on same-domain neighbours. Compared directly with whole-file code memory, the final method is ahead by 3.0 and 1.0 points, which is within the run-to-run variation of a single run; the evidence for the fix is the removal of the scaffold failures, not a higher total. Missing Vue imports remain (23 and 27 tasks with at least one affected component, against 17 and 19 for whole-file code memory): once the generator writes real pages instead of the scaffold, it has more opportunities for this slip.

[TODO: Status: manifest running (expected 10-16), seed 47 running, interleaved queued. Report whether the gain holds in all orders and whether the interleaved order is clearly worse; add one summary figure of the gain and CI per order, with a shared y-axis across the order plots.]

[TODO: Planned (existing data, final version): one complete case of transfer (failure, stored entry, why it was retrieved, injected text, change in code, outcome), a few lines; negative transfer is Figure 5.3.]

5.6.1 Fixing the Exposed Failure

The fix was developed on seed 11 while all other experiments used the frozen whole-file design. It changes only the code pool. Every entry type now shows a fragment rather than a file: on the seven seed-11 tasks where the whole-file design had returned the scaffold, accuracy rose from 3.6% to 45.2%, and no scaffold occurred. An entry is replaced only when its new example covers strictly more of the needed functions, so that the example does not switch back and forth. Whole-site failures create an entry after one occurrence, since waiting for a second meant that rare problems were learned after they had already done their damage, and three slots are allowed instead of two. Compile-error entries apply to every Vue site. Total accuracy on seed 11 is not used to judge these changes: memory chains diverge from the first task, and two runs of the same design can differ by about five points. This is debugging evidence; the full-benchmark evidence for the final method is the set of runs started after it was frozen (Table 5.5).

Chapter 6

Discussion

6.1 The Role of Memory Representation

Across the diagnostic and the full runs, this small generator responded more reliably to short code fragments than to accumulated textual lessons. Because the text and code designs differ in more than representation, this is a cross-version observation, not a controlled comparison. This agrees with Metis's finding that code memory complements text memory [5], and adds that for a 7B generator the format of the memory can decide whether it helps at all. The form of the code also matters: a fragment that looked like a complete file was copied as a finished answer, producing an untouched scaffold.

This contrasts with strong models, for which text memory transfers more reliably than code [5, 15]. The mechanisms differ (Metis executes its code; we show code to the generator), but together the results suggest that the best form of memory depends on the capability of the model that consumes it.

A second observation is that part of the gain is not "smarter" code but protection against collapse. In the earliest phase, short prompts made the model return the default scaffold in five of eight tasks, while longer, specific prompts did not. Both effects are real benefits of memory, but they should be described separately.

6.2 What Is Learned

The generator does not change; what changes is which reusable code is shown to it, and when. The system records which problems recur, associates them with code drawn from its own earlier successful and compiling websites, and retires lessons whose recurrence-based utility stays low. This is closer to automatically building a small set of preventive

code examples supported by observed prior outcomes than to acquiring new programming ability, and it is a property of the whole system, including a memory model larger than the generator. Within 101 tasks the memory stays small (a few code-pool entries and tens of active lessons); how it behaves over thousands of tasks, with many more problem types competing for three slots and older entries conflicting with newer ones, is not tested here.

6.3 Cost

The generator is small, but the system is not. Each Long Memory task takes about 12 minutes against about 6 for No Memory; the extra steps are checklist construction, applicability judgements and reflection by the 72B memory model, which we have not timed separately. The functional tester is shared by both arms and belongs to evaluation, not to deployment. If the fixed-prefix ablation shows that the learned fragments alone recover most of the gain, a deployed system could use the fragments without running the 72B model for every task, at the cost of losing online adaptation.

6.4 Limitations and Threats to Validity

- **Single runs.** WebGen-Bench itself reports single runs without error bars. We repeat No Memory under several orders to measure run-to-run noise (three runs completed, two running) and repeat Long Memory under additional orders with paired comparisons, but within each order Long Memory is still a single run. Two runs of the same Long Memory version can differ by about five points, more than No Memory varies, so the task-level intervals understate the full uncertainty.
- **One generator and one scaffold.** All results are for WebGen-LM-7B with Bolt.diy and a Vue template. The finding that code is followed where text is not may be specific to this model rather than to small models in general; larger generators may follow text well and benefit less from code fragments.
- **Residual generator errors.** About 4.6% of generated components omit a Vue import with or without memory; examples did not reduce it, and the protocol forbids repairing it after generation.

- **Code pool as a learned prefix.** Because its triggers are general, the code pool behaves partly like a fixed prompt prefix; the fixed-prefix ablation (Section 5.5) measures how much of the gain this explains.
- **Rare problems are learned late.** A safeguard exists only after a problem has occurred, so memory helps frequent problems more than rare ones, and with three slots several problem types can compete for space (Section 5.6.1).
- **Appearance grader.** We grade appearance with Qwen2.5-VL-32B instead of GPT-4o, so appearance scores are compared only between our arms. On tasks where both arms produce a working site, mean appearance is similar (3.25 vs 3.22); this subset is selected by the outcome, so it does not establish that memory has no effect on appearance.
- **Iterative development on the same benchmark.** This is the most serious threat. Every version was designed after looking at failures on the same 101 tasks on which it is evaluated. Reflection rules teach a method rather than answers, every run starts from an empty memory, and the final version was frozen before its runs on all orders except seed 11, but these facts limit the risk without removing it. The results should be read as a within-benchmark study; held-out evaluation is future work (Chapter 7).
- **Evaluator noise.** GUI testers can exhaust their interaction budget or misjudge pages [37]. We keep raw evaluator transcripts and distinguish interaction-budget exhaustion from logic failures. Their judgements were not checked against a human reader.
- **Memory model stronger than generator.** The 72B memory model is larger than the 7B generator, so part of the gain could be the larger model solving the task for the small one. Three observations argue against this reading. The checklist only restates the requirement, and on its own its gain has an interval that includes zero (Table 5.4). Every code fragment is copied verbatim from a website that the 7B generator itself built earlier, and code checks reject lines that are not in that file. And the memory model never sees the current task's evaluation. The results therefore show what a small generator gains when a stronger model organises its own past experience, not self-improvement in isolation. [TODO: Status: fixed-prefix run queued after seed 47. State how much of the gain remains without accumulated memory and without the 72B model at generation time.]

6.5 Ethical Considerations and Impact

Data and privacy. The benchmark requirements are synthetic and contain no personal data. All models run on university hardware; no task data were sent to external services.

Environmental cost. A full run takes about a day on two H200 GPUs (Section 3.5); we limited runs by testing changes on three to five tasks and checking code statically before evaluation.

Self-improving code agents. A memory that turns observed outcomes into future behaviour can also entrench mistakes or unsafe patterns. Our design keeps full provenance of every memory, requires code examples to compile, and never edits the source record, so that any injected content can be traced back to the task that produced it. Deployed systems would additionally need review of stored code for security.

Economy and society. Tools that let small, locally hosted models build working websites lower the cost of software creation and keep data on local infrastructure, which may benefit small organisations. They may also reduce demand for routine front-end work. Generated websites still require human review for correctness and accessibility.

Use of AI tools. AI tools were used to assist with programming the experimental pipeline and analysis scripts, and to provide comments on drafts. All text was written by the author, who decided which suggestions to adopt. All numbers were computed by the scripts in the public repository and checked by the author. [TODO: Author: this statement holds only after the final text has been rewritten in your own words; check it against what was actually done before submission.]

Research integrity. We report negative results (text memory's decline, the scaffold failure of whole-file examples) and invalidated our own earlier results when a delivery bug was found.

6.6 Project Management

The project was organised in versions. The cluster ran one frozen version at a time; new ideas were tested locally on small samples and with static code checks, and a version was

replaced only when the new one was shown to be better. Decisions and their evidence were recorded continuously in a decision log, which made it possible to reconstruct why each component exists.

Chapter 7

Conclusion

7.1 Answers to the Research Questions

RQ1 A qualified yes, for WebGen-LM-7B with Bolt.diy on WebGen-Bench. In a single run over 101 tasks whole-file code memory improves accuracy from 28.5% to 40.0% (paired gain 11.5 points, 95% CI 6.0–16.9) and reduces zero-score tasks from 30 to 13 under the official protocol, without changing the generator or the evaluation. The gain is in functional accuracy; much of it comes from sites that now render at all. [TODO: Status: replace with the final method on the manifest order (running, expected 10-16), and summarise all orders once seed 47 and interleaved finish.]

RQ2 In this system, the small generator followed short code fragments copied from its own earlier websites. A requirement checklist alone reduced whole-site failures but added only 3.4 points of accuracy (interval includes zero), while memory added 8.1 points on top of it (interval excludes zero). Textual lessons were rarely followed in diagnostics, and the text-dominated version fell behind No Memory late in the sequence; because the versions differ in more than the form of memory, this is not a controlled comparison of text and code. Fragments must be partial: a complete file was copied as a finished answer. [TODO: Status: add the fixed-prefix result (queued after seed 47).]

RQ3 The advantage of code memory grows over the sequence (40.4% vs 29.5% on tasks 31–101, 38.3% vs 26.0% on the last 35), while text memory falls below No Memory after task 30, consistent with, but not proof of, learning from accumulated experience; the fixed-prefix ablation tests the alternative of a fixed prompt. In two shuffled orders the gain of whole-file code memory fell to about 4.5 points because one code memory, a

whole-file example, made the generator return the scaffold; with examples reduced to verified fragments the scaffolds disappeared and the gain was 7.4 points with domains adjacent (interval excludes zero) and 5.5 points with domains mixed (interval includes zero). Positive gains were observed in both kinds of order, but robustness to order is not yet established. [TODO: Manifest, seed 47 and interleaved order with the final method.]

7.2 Future Work

Two further experiments would address the main remaining questions:

- **Held-out evaluation.** Because the method was developed while observing results on all 101 tasks, a clean test needs tasks that were never seen: a held-out domain, or a second website-generation benchmark, with the method frozen beforehand.
- **Controlled form-of-memory comparison.** Keep retrieval, trigger and token budget fixed and vary only the form: textual rule, pseudocode, short fragment, complete file, plus random-code and length-matched text controls.

Further directions are repeating each order more than once, to separate order effects from run-to-run variation, and testing larger generators of the same family, with a text-memory arm, to see whether the preference for code depends on model size.

Bibliography

[1] Shuai Bai et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025. URL: https://arxiv.org/abs/2502.13923.

[2] Jianlv Chen et al. M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024. URL: https://arxiv.org/abs/2402.03216.

[3] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023. URL: https://arxiv.org/abs/2304.05128.

[4] Prateek Chhikara et al. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2504.19413.

[5] Zijie Dai et al. Metis: Bridging text and code memory for self-evolving agents. *arXiv preprint arXiv:2606.24151*, 2026. URL: https://arxiv.org/abs/2606.24151.

[6] Xiang Deng et al. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023. URL: https://arxiv.org/abs/2306.06070.

[7] Shehzaad Dhuliawala et al. Chain-of-verification reduces hallucination in large language models. *arXiv preprint arXiv:2309.11495*, 2023. URL: https://arxiv.org/abs/2309.11495.

[8] Yufeng Du, Minyang Tian, et al. Context length alone hurts LLM performance despite perfect retrieval. In *Findings of EMNLP*, 2025. arXiv:2510.05381. URL: https://arxiv.org/abs/2510.05381.

[9] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. URL: https://doi.org/10.1201/9780429246593.

[10] Keno Harada et al. Curse of instructions: Large language models cannot follow multiple instructions at once. *arXiv preprint*, 2025. URL: https://openreview.net/forum?id=R6q67CDBCH.

[11] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of ACL*, 2024. URL: https://arxiv.org/abs/2401.13919.

[12] Binyuan Hui et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. URL: https://arxiv.org/abs/2409.12186.

[13] Daniel Jaroslawicz et al. How many instructions can LLMs follow at once? *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2507.11538.

[14] Carlos E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL: https://arxiv.org/abs/2310.06770.

[15] Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents. *arXiv preprint arXiv:2604.14004*, 2026. URL: https://arxiv.org/abs/2604.14004.

[16] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL: https://arxiv.org/abs/2309.06180.

[17] Yuetai Li et al. Small models struggle to learn from strong reasoners. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2502.12143.

[18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. URL: https://arxiv.org/abs/2307.03172.

[19] Zimu Lu et al. WebGen-Bench: Evaluating LLMs on generating interactive and functional websites from scratch. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2505.03733.

[20] Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL: https://arxiv.org/abs/2303.17651.

[21] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *Proceedings of ICLR*, 2026. arXiv:2509.25140. URL: https://arxiv.org/abs/2509.25140.

[22] Charles Packer et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL: https://arxiv.org/abs/2310.08560.

[23] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of UIST*, 2023. URL: https://arxiv.org/abs/2304.03442.

[24] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of ICML*, 2023. URL: https://arxiv.org/abs/2302.00093.

[25] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL: https://arxiv.org/abs/2303.11366.

[26] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, and Yu Su. AgentCL: Toward rigorous evaluation of continual learning in language agents. *arXiv preprint arXiv:2606.02461*, 2026. URL: https://arxiv.org/abs/2606.02461.

[27] Chenglei Si et al. Design2Code: Benchmarking multimodal code generation for automated front-end engineering. *arXiv preprint arXiv:2403.03163*, 2024. URL: https://arxiv.org/abs/2403.03163.

[28] Daegyu Sung, Yukyeong Lee, Geon Park, Yumin Choi, and Sung Ju Hwang. Super library agent: Joint generation and maintenance of multiple applications beyond the single codebase. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. arXiv:2608.29310. URL: https://arxiv.org/abs/2608.29310.

[29] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of EACL*, 2026. URL: https://arxiv.org/abs/2504.07952.

[30] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024. URL: https://arxiv.org/abs/2404.13208.

[31] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL: https://arxiv.org/abs/2305.16291.

[32] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of ICML*, 2025. URL: https://arxiv.org/abs/2409.07429.

[33] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, et al. Evo-memory: Benchmarking LLM agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025. URL: https://arxiv.org/abs/2511.20857.

[34] Andreas Zeller. *Why Programs Fail: A Guide to Systematic Debugging*. Morgan Kaufmann, 2 edition, 2009. URL: https://doi.org/10.1016/B978-0-12-374515-6.X0000-7.

[35] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *Proceedings of ICLR*, 2026. arXiv:2510.04618. URL: https://arxiv.org/abs/2510.04618.

[36] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. URL: https://arxiv.org/abs/2308.10144.

[37] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023. URL: https://arxiv.org/abs/2306.05685.

[38] Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *Proceedings of ICLR*, 2024. URL: https://arxiv.org/abs/2306.07863.

[39] Huichi Zhou et al. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2508.16153.

[40] Shuyan Zhou et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. URL: https://arxiv.org/abs/2307.13854.